

A2 - Hadoop and MapReduce

Arnab Kumar Ghosh

February 22, 2026

1 Part 1: Hadoop Fundamentals

1.1 Task 2: WordCount in Standalone Mode

Q2.1: What files were created in the output directory? What does each file represent and why are they named that way?

The following files were generated in the output directory after running the WordCount MapReduce job in standalone mode:

- **_SUCCESS**: It is an empty file. It is actually a marker file indicating the job completed without errors.
- **part-r-00000**: The actual result file. The name follows a convention used by Hadoop for output files:
 - **part**: Indicates it is a segment of the total output.
 - **r**: Signifies it was produced by a reducer (as opposed to *m* for mappers).
 - **00000**: Represents the reducer number (0-indexed). It is the 0-indexed ID of that reducer. In this case, since we only have one reducer, it is named 00000. If there were multiple reducers, we would see files like **part-r-00001**, **part-r-00002**, etc., each containing the output from a different reducer.

Q2.2: How many times did the word “Discovery” (case-sensitive) appear in the text? ? Show/write the command you used to find this.

The word “Discovery” (case-sensitive) appeared **5** times in the text. The following command was used to retrieve this result from the output file:

```
grep "Discovery" ./output/part-r-00000
```

Q2.3: Explain the difference between Hadoop’s Standalone Mode and Pseudo-Distributed Mode. When would you use each mode? Give a specific scenario for each

Hadoop supports different execution modes depending on the infrastructure and use case. The two modes relevant to this assignment are:

- **Standalone Mode:** Runs as a single Java process using the local filesystem. There is no NameNode or DataNode in this mode. Scaling is not possible in this mode.

Scenario: This mode is usually used during initial development or debugging to test if the MapReduce logic is correct. This mode is also useful for learning and experimenting with Hadoop without needing to set up a full cluster. For example, if I am writing a new Mapper or Reducer class, I can quickly run it in standalone mode to verify that it produces the expected output on a small test dataset before deploying it to a larger cluster.

- **Pseudo-Distributed Mode:** Simulates a cluster on a single machine by running multiple Java processes. This mode runs NameNode, DataNode as Hadoop daemons. This mode uses HDFS instead of the local filesystem.

Scenario: This mode is used to test HDFS configurations and ensure the code works in a distributed environment before deploying to many physical nodes. For example, if I want to test how my MapReduce job interacts with HDFS, I can set up a pseudo-distributed mode on my local machine. This allows me to verify that the job can read from and write to HDFS correctly. I can also verify that the data is being processed as expected in a distributed manner, even though it is all running on one machine.

1.2 Task 3: Configure Pseudo-Distributed Mode

Q3.1: What are the roles of the configuration files `core-site.xml` and `hdfs-site.xml`?

Hadoop uses XML files to define its behavior and environment.

- **core-site.xml:** This file contains common settings that apply to all Hadoop components, such as the default filesystem URI. For **fs.defaultFS** I configured this to **hdfs://localhost:9000**. This tells the Hadoop client that the default filesystem is HDFS and that the NameNode is running on the local machine at **port 9000**. It is necessary because, without it, Hadoop would default to using the local Linux filesystem .
- **hdfs-site.xml:** This file is specific to HDFS and controls how the distributed filesystem stores and replicates data. For **dfs.replication** I set this to **1**. This determines how many copies of each data block are stored across the cluster. Since we are only using one VM, we cannot have more than one copy, so setting this to 1 is necessary to avoid "under-replicated" errors.

Q3.2: Describe the roles of the NameNode and DataNode daemons. How do they work together to store and retrieve files? What would happen if the NameNode crashed in a real cluster?

The HDFS architecture operates on a Master-Slave relationship to manage distributed data storage:

- **NameNode (Master):** Acts as the filesystem manager. It does not store the actual data of the files but maintains the metadata. Metadata includes the directory tree, file-to-block mapping, and the specific locations of those blocks.

- **DataNode (Slave):** Serves as the worker daemon. Its primary responsibility is to store the actual raw data blocks on the local disk of the machine it resides on.
- **How NameNode and DataNode work together::** When a client needs to access a file, it first queries the NameNode to determine which DataNode holds the required blocks. The client then communicates directly with those DataNodes to retrieve or write the actual data.
- **If NameNode crashed:** In a real cluster, if the NameNode crashes, the entire cluster becomes unavailable. This is because the NameNode holds the only map of the data (metadata), and the DataNodes cannot reconstruct files from the raw blocks alone. To solve this problem, “High Availability” setups are used to maintain a standby NameNode.

Q3.3: Explain what HDFS is and how it differs from the local filesystem on your virtual machine. Why would a data engineer choose to use HDFS instead of just storing files locally? ? Give a specific scenario.

The primary differences and advantages of HDFS compared to a local filesystem are outlined below:

- **Architectural Difference:** A local filesystem (such as Ext4) is managed by the Operating System and is limited to the storage capacity of a single physical disk. In contrast, HDFS is a distributed filesystem that sits “on top” of the host OS. It allows to span across thousands of machines while presenting them as one single, massive storage pool.
- **Why Choose HDFS?:** It is designed specifically for big data workloads. It provides high-level fault tolerance through data replication and enabling parallel processing of massive datasets.
- **Industry Scenario:** For example, if I am working as a data engineer at a firm that processes 100 TB of log data. A single server cannot physically hold 100 TB, and processing it on one machine would take several days. By using HDFS, I can distribute that data across 50 servers and replicate the data blocks across multiple nodes. This approach reduces the processing time and increases fault tolerance. If multiple servers fail, HDFS automatically uses the replicated copies stored on other healthy nodes to ensure no data loss occurs. This allows MapReduce jobs to continue running without interruption.

1.3 Task 4: WordCount with HDFS and Custom Code

Q4.1: Explain the roles of the different classes in the file WordCount.java

The `WordCount.java` file consists of three primary components that define the MapReduce job workflow:

- **WordCount (Main Class):**

- This acts as the driver for the application.
- It configures the MapReduce job, setting the Mapper, Reducer, and Combiner classes, as well as defining the input and output paths in HDFS.
- **TokenizerMapper (Inner Class):**
 - This class extends the **Mapper** base class.
 - Its role is to process input text line-by-line, breaking strings into individual tokens (words) using a **StringTokenizer**.
 - For every word found, it emits a key-value pair consisting of the word as the key and the number 1 as the value: **(word, 1)**.
- **IntSumReducer (Inner Class):**
 - This class extends the **Reducer** base class.
 - It receives the intermediate keys (words) and a list of all the “1”s associated with that word from the Shuffle & Sort phase.
 - Its role is to iterate through these values, sum them up, and emit the final count for each unique word: **(word, total)**.

Q4.2: What is HDFS, and how is it different from the local filesystem on your virtual machine?

HDFS (Hadoop Distributed File System) is a specialized filesystem designed to store very large datasets across a cluster of machines. Its differences from the local VM filesystem are:

- **Storage Scope:** The local filesystem is limited to the storage of a single virtual machine. HDFS is a distributed system that can span across thousands of physical servers, aggregating their disks into one large pool.
- **Fault Tolerance:** Local storage is susceptible to data loss if the VM or its disk fails. HDFS automatically replicates data blocks across different nodes (typically with a replication factor of 3 in production) to ensure high availability.
- **Data Locality:** Unlike a local filesystem where data must be moved to the CPU, HDFS allows Hadoop to move the computation to where the data is stored. This is essential for processing large amounts of data efficiently.

1.4 Task 5: Modified WordCount (First Letter Counter)

The original `WordCount.java` was modified into `FirstLetterCount.java` to count how many words begin with each letter of the alphabet (case-insensitive). The Mapper was changed to extract only the first character of each token, convert it to lowercase, and emit **(letter, 1)**. The Reducer remains unchanged, summing the counts per key.

Analysis: The most frequent starting letters are **t** (18,613), **a** (12,893), and **o** (9,720), while the least frequent are **x** (32), **z** (55), and **q** (173). This closely matches expectations for English text. Because common words such as “the”, “that”, “to”, and “a” all begin with *t* or *a*. On the other hand, very few English words start with *x*, *z*, or *q*. This shows that a small number of very common words have a big effect on which letters appear most often.

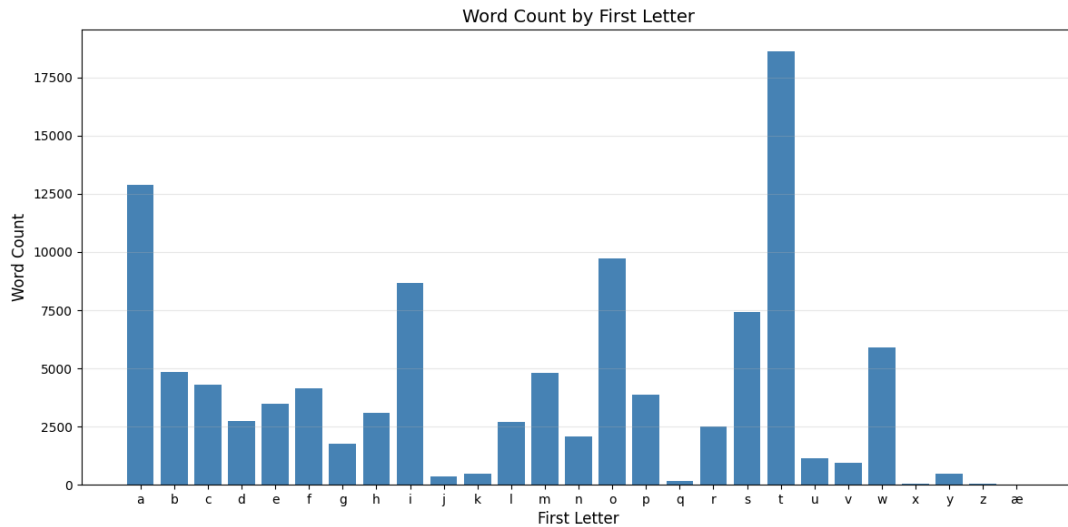


Figure 1: Distribution of word counts by first letter (a–z) in the input text.

1.5 Task 6: SQL and NoSQL Concepts

Q6.1: Data Structure Classification

Classification: Semi-structured data

Justification: In my understanding, JSON tweets are semi-structured data because they have tags (keys) and markers that explain what each part means. This makes the data self-describing. They have fields like id, user, and text, so they look organized. But they do not follow a fixed structure like relational databases. Each tweet can have different fields. Some of the tweets have **retweeted_status** fields, and others do not. They can also contain nested objects, like a user object inside a tweet. So, they are structured, but not strictly structured.

Q6.2: SQL vs. NoSQL Analysis

1.5.1 1. SQL Databases (Relational)

Pros:

- **ACID Properties:** This guarantees Atomicity, Consistency, Isolation, and Durability. These rules make sure database transactions are safe and reliable.
- **Standardization:** Relational database uses a structured query language (SQL). SQL is a common and widely known language. It supports complex queries, joins and relationships.
- **Data Integrity:** They use strict schemas. They also use rules like foreign keys and unique constraints. This prevents wrong or invalid data from being stored.

Cons:

- **Rigid Schema:** Changing the database structure can be difficult. It may require system downtime. This makes it less flexible for rapidly evolving data models.

- **Vertical Scaling:** Typically requires scaling up through buying a bigger server. But there is a limit to how much hardware can be upgraded.

Real-World Use Case: In a SaaS-based tour management solution I built before, we used **PostgreSQL** as the main database. The application handled critical financial transactions, such as booking confirmations and payment processing. After payment confirmation, the application had to atomically update booking statuses, send accounting entries in external systems (Fortnox), and calculate future payment deadlines. This scenario demanded strong Consistency and Availability (CA in the CAP theorem) to ensure financial data was always accurate and synchronized.

1.5.2 2. NoSQL Databases (Non-Relational)

Pros:

- **Schema Flexibility:** No pre-defined structure is needed, so we can store unstructured or semi-structured data. This makes it easy to change or update the database quickly as the application evolves.
- **Horizontal Scalability:** Can grow by adding more servers instead of just upgrading one machine. This makes it easier to handle very large amounts of data.
- **Performance:** Can handle a lot of reads and writes quickly because they do not always enforce strict consistency. This is called eventual consistency.

Cons:

- **Eventual Consistency:** Data may not be immediately consistent across all nodes. This can be a problem for critical financial transactions.
- **Limited Queries:** Generally lacks the sophisticated join capabilities and standardized query language of SQL.

Real-World Use Case: In the same tour management ecosystem, we implemented a microservice using **MongoDB** to aggregate flight and yacht booking data from various third-party airlines and yacht companies. The data schemas from these external providers were massive and the schemas were all different. So the data acts as unstructured or semi-structured data streams relative to our core application. A NoSql database like MongoDB allowed us to handle this heterogeneous data efficiently without breaking the schema. This microservice was designed for high throughput and could tolerate eventual consistency. It was primarily used for analytics and reporting rather than critical transactions.

1.5.3 3. Trade-offs and Decision Criteria

The decision between SQL and NoSQL often depends on the CAP theorem trade-offs:

- We will choose **SQL** when data integrity and strict consistency are the main concerns. If the application involves complex relationships (joins) or financial transactions where an ACID property is required, SQL is the best choice.
- We will choose **NoSQL** when the data structure is evolving, semi-structured or unstructured. If the application requires massive horizontal scaling to handle high velocity/volume then NoSQL is preferred. It is the best choice for content management, real-time analytics, or ingesting diverse API data.

- **Hybrid Approach:** As I demonstrated in the SaaS example, modern architectures often use both: SQL for core transactional logic and NoSQL for handling flexible, high-volume data ingestion in specific microservices.

2 Part 2: Advanced Topics (Task 7)

2.1 Methodology

The MapReduce pipeline was implemented in Python using Hadoop Streaming API. The **mapper** reads one JSON tweet object per line from standard input. Then it parses the JSON string and converts it into a Python dictionary object. After that it checks for the `retweeted_status` field exists in the object or not. If the field exists, the tweet is a retweet and is skipped entirely to avoid double-counting. For original tweets, the tweet text is extracted and converted to lowercase. Then words are splitted using a regex pattern `(\b\w+\b)` to strip punctuation. For each of the target pronouns (*han*, *hon*, *den*, *det*, *denna*, *denne*, *hen*), the mapper checks if the pronoun appears among the extracted words. If found, it emits `pronoun\t1`. Each distinct pronoun is counted at most once per tweet, but multiple different pronouns in the same tweet are each counted separately. The **reducer** aggregates the counts by summing all values for each pronoun key and produces the final results.

2.2 Results

The total number of pronoun instances across the entire dataset is **2,708,836**.

Pronoun	Count	% of Instances
den	1,220,721	45.06%
denna	22,471	0.83%
denne	3,986	0.15%
det	456,521	16.85%
han	666,495	24.60%
hen	31,108	1.15%
hon	307,534	11.35%

Table 1: Swedish pronoun distribution across the tweet dataset.

2.3 Analysis

The most common pronoun is **den** (45.06%), followed by **han** (24.60%) and **det** (16.85%). On the other hand, the least common are **denne** (0.15%) and **denna** (0.83%). The dominance of *den* and *det* is expected, as these are the Swedish equivalents of the definite articles “the” and “it/that”(I used google translate to get the meaning in English). These two are the most frequently used words in everyday Swedish. The traditional gendered pronouns *han* appears more than twice of *hon*. This may reflect the demographics or topics of the Twitter users in the dataset. Notably, the gender-neutral pronoun **hen** accounts for only 1.15% of all instances, which is significantly lower than traditional pronouns.

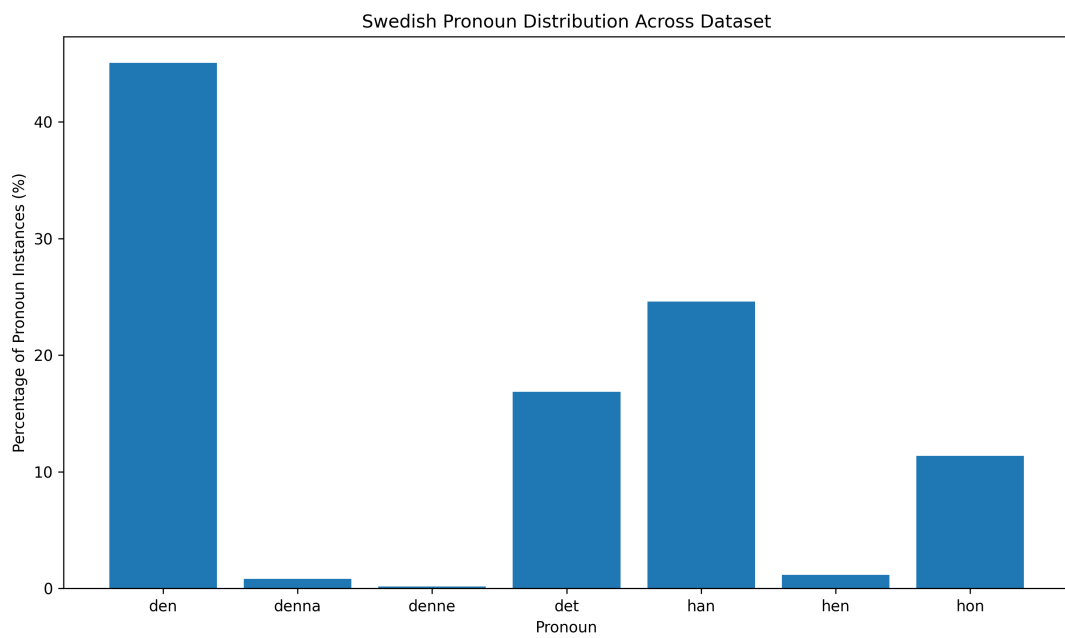


Figure 2: Bar chart showing the percentage distribution of Swedish pronouns.